

Split(NotSo)Wise

Кратка Документация

Ирена Станчева - 4MI0600336

1. Въведение в SplitWise

Системата SplitWise е клиент-сървър приложение, базирано на Java, за управление на споделени разходи между приятели и групи. Системата използва за комуникация между сървър и клиент команди с имплементация на паралелен сървър.

2. Архитектура и основни компоненти в решението

2.1. Диаграма на архитектурата

Клиент → TCP/NIO → Сървър: Selector + Thread Pool → Command Processing → Бизнес логика → Data Storage

2.2. Client-Server метод

В src/ директорията имаме следните MAIN методи за стартиране на server и client: ServerMain, ClientMain. Тези методи извикват съответните start() методи на клиента и сървъра. Протокола, по който се извършва мрежовата комуникация е TCP чрез NIO комуникация.

Клиент:

SplitWiseClient - Обработва потребителски вход/изход и мрежова комуникация със сървъра. След като получи от потребителя вход, изпраща го на сървъра и чака отговор от него. След като получи успешно отговор уведомява потребителя със съобщение дали успешно или неуспешно е била извършена желаната заявка.

Сървър:

SplitWiseServer - Обработва клиентски заявки, използвайки NIO и обединяване на нишки. Чака да получи от клиент заявка за свързване и обработка на потребителска заявка. При получена успешно потребителска заявка, изпраща Input-а на CommandExecutor, който от своя страна извършва дадената функционалност и връща съобщение дали е била успешна. Сървърът изпраща получения отговор на клиента и цикъла се повтаря.

Сървърът менажира клиентските заявки чрез ClientRequestHandler, който от своя страна менажира клиентските сесии (Session) чрез ClientID.

2.3. Команден слой

CommandCreator: Разделя потребителския вход в структурирани команди с име на командата и аргументи.

CommandExecutor: Насочва командите към съответните изпълнители на бизнес логиката.

Executors: Отделни класове за всеки тип команда (RegistrationExecutor, LoginExecutor и др.)

2.4. Слой на бизнес логиката

SplitWiseRepository: Главна фасада на бизнес логиката.

UserRepository: Управлява запазването на потребителските данни във файлове и в структури от данни, от които лесно да се вземат при нужда.

Processors: Специализирани класове за различни операции (LoginProcessor, RegisterProcessor и др.).

2.5. Модел на данни

Потребител: Представлява потребители на системата с потребителски име, парола, съобщения при влизане в системата, приятели и групи.

Група: Управлява членството в група. Членовете в една група са приятелите на потребителя, който я е създал

Приятел: Представлява приятелски отношения - приятел на потребител с дълговете, които си дължат.

Дълг: Проследява финансовите задължения между потребителите - сума и причина за дълга.

История на плащанията: Съхранява историята на платените (split) дългове от потребителя, за да се запишат те в двоичен файл с историите на плащанията.

Нотификации: Грижи се всеки път, когато приятел на потребител извърши транкация на дълг да я запази тук и при следващо успешно вписване в системата от потребителя да го уведоми за нови плащания. След това тези нотификации се трият, за да се освободи място за нови.

Заявки: Заявки за изпращане на данни при split операция, за да не се разнасят из методите по 5-6 параметъра, а да се обвият в един.

2.6. Валидационен слой

SplitWiseValidator: Валидиране на бизнес правила

ExecutorValidator: Валидиране на параметри на команда

UserValidator: Валидиране на потребителски данни

2.7. Изключения

В пакета exceptions се намират всички видове изключения, които биха могли да се хвърлят някъде из бизнес логиката.

3. Concurrent изпълнение

3.1. Управление на нишки

Сървър: Използва pool от нишки с фиксиран размер
(Executors.newFixedThreadPool(MAX_EXECUTOR_THREADS))

Клиент: Едонишков с блокиращ вход/изход

Гаранция за паралелизъм: Нишкобезопасни структури от данни и синхронизирани методи

3.2. NIO имплементация

Модел на селектор: SplitWiseServer използва NIO селектор за ефективен вход/изход

Неблокиращи канали: ServerSocketChannel и SocketChannel със събития OP_ACCEPT/OP_READ

Управление на буфера: Разпределение на ByteBuffer (1024 байта) за трансфер на данни

3.3. Механизми за безопасност на нишките

Изолиране на данни:

```
// Всеки клиент получава изолирано хранилище
private final Map<String, SplitWiseRepository> clientRepositories;
private final Map<SocketChannel, String> clients;
private final Map<SocketChannel, Session> sessions = new
ConcurrentHashMap<>();
```

Точки на синхронизация:

- **Файлови операции:** синхронизирани методи в UserRepository
- **Управление на сесии:** ConcurrentHashMap за безопасно съхранение на сесии
- **Обработка на команди:** Изпращане на задача към изпълнител на пул от нишки

Concurrent структури от данни:

- **ConcurrentHashMap:** За споделени карти клиент/сесия
- Локални за нишките хранилища за изолирани потребителски данни

3.4. Управление на сесии

Изолиране на сесии: Всеки клиент поддържа независима сесия

Login в сесия: Проследява се чрез обект на сесия с потребителско име

Cleanup: Правилно освобождаване на ресурси при прекъсване на връзката на клиента чрез команда logout

4. Съхранение на данни

4.1. Файлово съхранение

Потребители: Сериализирани в splitwise_users.bin

История на плащанията: Сериализирани в payment_history_for_users.bin

Атомни актуализации: Файловите операции са обвити в синхронизирани блокове.

4.2. Класове на процесори

RepositoryProcessor: Общи файлови входно-изходни операции (Generic методи)

UserProcessor: Специфична за потребителя логика за съхранение

PaymentHistoryProcessor: Управление на историята на плащанията